

Using DMAIC to Reduce Negative Player Reviews in Dead by Daylight

Jerry Shi

1 Course Scenario and Problem

For this project, I chose *Dead by Daylight*, an online multiplayer horror game where four survivors play against one killer. The game depends on servers, matchmaking, character abilities, frequent updates, and downloadable content. This creates many possible quality problems. In this case study, I am acting as a game quality analyst.

The problem is that the percentage of negative Steam reviews is fairly high and changes a lot from week to week. Steam is an online platform where people can buy, play, and review video games. I treated a negative review as a nonconforming unit because the player did not recommend the game. Reducing the problems behind those reviews could improve player satisfaction, keep more players active, improve the game's reputation, and help future sales.

2 Define

The scope is English Steam reviews written from April 14, 2025 through November 9, 2025. The reviews are used as a measurable signal of player satisfaction. They cannot prove that one patch caused every complaint, and some reviews are based mostly on personal preference.

The main critical-to-quality characteristic, or CTQ, is the weekly proportion of negative reviews:

$$p_i = \frac{\text{negative reviews in week } i}{\text{total reviews in week } i}$$

A smaller value is better. The second CTQ is the weekly actionable-complaint proportion. It counts reviews that name a specific issue, such as balance, bugs, performance, servers, matchmaking, cheating, or monetization.

Table 1: CTQ characteristics

CTQ	Definition	Direction
Weekly negative-review proportion	Negative reviews divided by all reviews in the week	Smaller is better
Weekly actionable-complaint proportion	Reviews naming a specific issue divided by all reviews in the week	Smaller is better

I am not trying to remove every negative review. The current average negative-review proportion is about 24.7%. A reasonable first goal is to lower it below 20% within 12 weeks and avoid special-cause spikes above the upper control limit.

2.1 SIPOC Diagram

Table 2: SIPOC summary

Part	Details
Suppliers	Players, QA testers, developers, server provider, and Steam
Inputs	Reviews, bug reports, game code, server data, and patch schedule
Process	Collect, classify, reproduce, prioritize, fix, test, release, and monitor
Outputs	Resolved issues, stable patches, patch notes, and lower negative-review rate
Customers	Current players, future players, support team, and development team

The table lists the SIPOC parts. The SixSigma process map below shows how the main inputs move through the complaint-response process and become outputs for players and the development team.

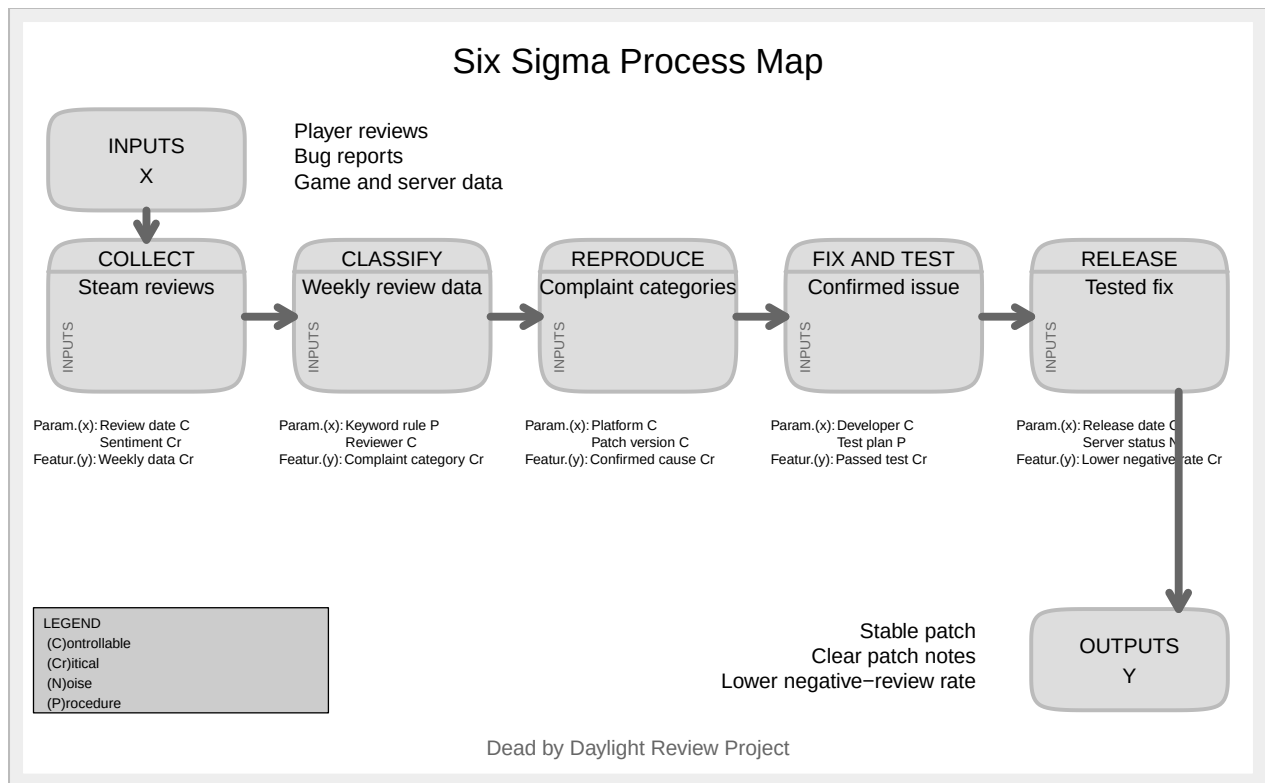


Figure 1: SIPOC process map made with the SixSigma package.

2.2 Process Flowchart

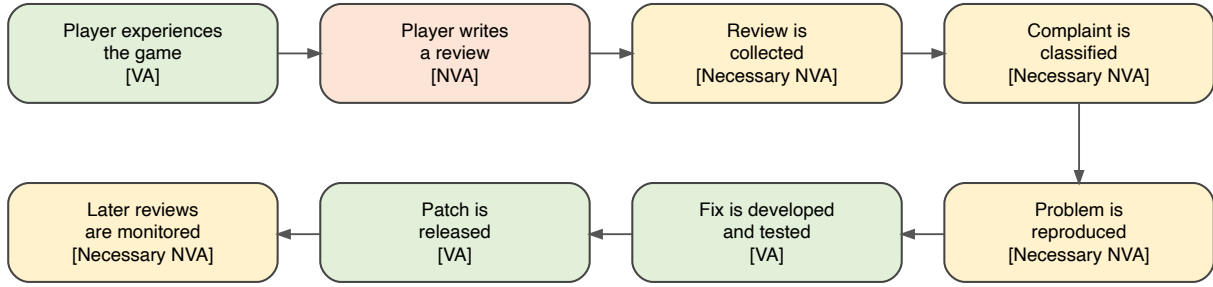


Figure 2: Review-response flowchart. VA means value-added, necessary NVA means required but not directly value-added, and NVA means non-value-added.

The main value-added steps are developing, testing, and releasing a useful fix. Collecting, classifying, reproducing, prioritizing, and monitoring are necessary but do not directly change the game. Unclear reviews and repeated waiting are non-value-added.

2.3 DMAIC Tollgates

Table 3: DMAIC tollgates

Stage	Tollgate
Define	Problem, scope, CTQs, SIPOC, and process map are agreed on
Measure	Dates and sentiment are checked; weekly data and check sheet are ready
Analyze	Unusual weeks and main complaint categories are reviewed
Improve	Owners are assigned and proposed changes are tested
Control	The chart, demerit score, and response plan are used each week

The Define tollgate is passed when the problem, goal, scope, CTQs, SIPOC, and process steps are agreed on.

3 Measure

The full file contains 277,479 reviews. The selected 30-week period contains 30,782 reviews, including 7,599 negative reviews. The overall negative-review proportion is 24.7%.

The weekly number of reviews changes, so I used a p -chart instead of an np -chart. A p -chart allows a different sample size each week. The negative-review count is the data and the total number of reviews is the sample size.

The Measure tollgate is passed after the date and sentiment columns are checked, the weekly groups are created, and the complaint check sheet is finished.

3.1 Weekly p-Chart

```
## List of 11
## $ call      : language qcc(data = weekly$negative,      type = "p", sizes = weekly$total, ..
## $ type      : chr "p"
## $ data.name  : chr "weekly$negative"
## $ data       : num [1:30, 1] 168 133 99 128 142 124 180 173 134 488 ...
## $ statistics: Named num [1:30] 0.242 0.229 0.218 0.233 0.214 ...
## $ sizes      : num [1:30] 694 582 455 550 664 ...
## $ center     : num 0.247
## $ std.dev    : num 0.431
## $ nsigmas    : num 3
## $ limits     : num [1:30, 1:2] 0.198 0.193 0.186 0.192 0.197 ...
## $ violations:List of 2
```

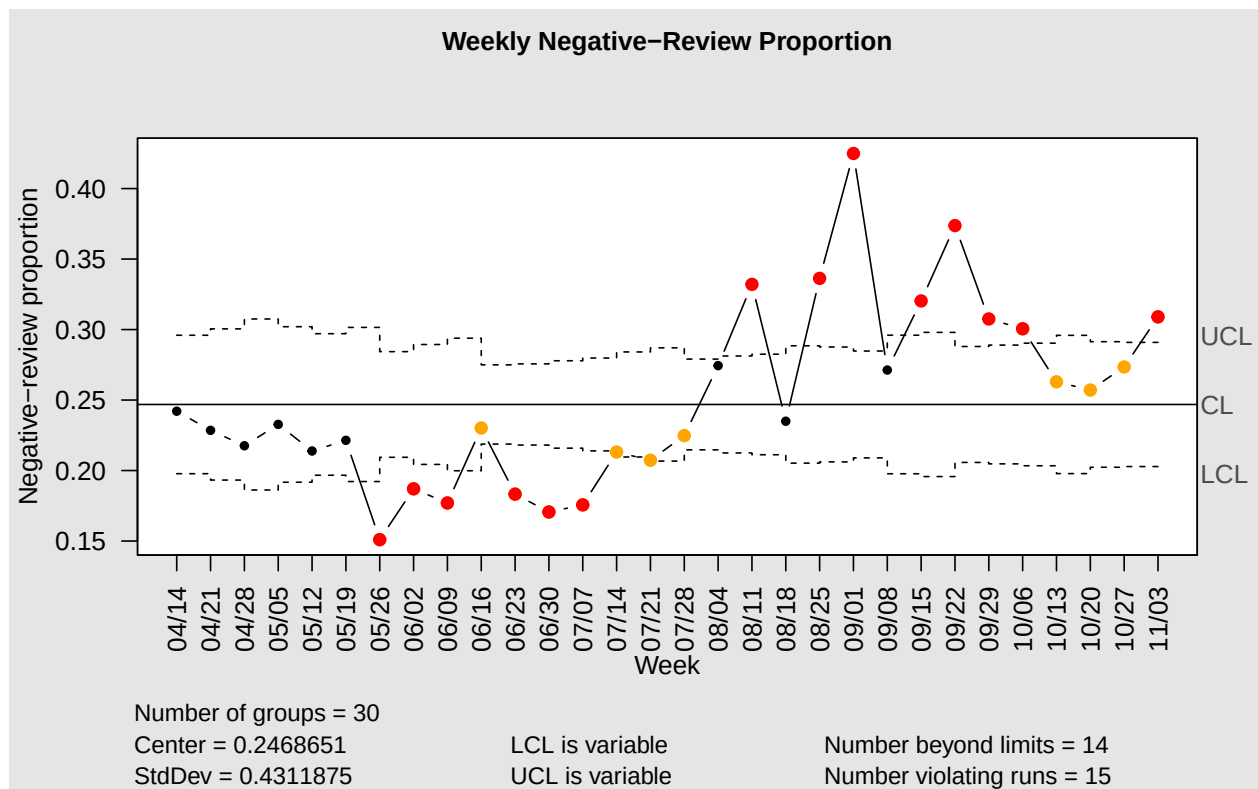


Figure 3: Weekly p-chart for the proportion of negative reviews.

3.2 Check Sheet

I changed the review text to lowercase and used simple keyword rules to place each negative review into one complaint category. Each review was assigned to the first category that matched. Reviews without a clear keyword were placed in Other/unclear. This is less accurate than reading every review by hand, but it creates one consistent check sheet.

Table 4: Complaint check sheet

Complaint	Count	Percent
Other/unclear	2533	33.3
Gameplay/balance	1717	22.6
Technical bugs/performance	1067	14.0
General enjoyment/design	609	8.0
Updates/developer decisions	421	5.5
Server/connection	369	4.9
Monetization	333	4.4
Matchmaking	244	3.2
Toxicity/community	170	2.2
Cheating	92	1.2
Privacy/data	44	0.6

4 Analyze

The p -chart center line is 0.2469. There are 14 weeks outside the three-sigma limits: 8 above the UCL and 6 below the LCL. The highest weekly negative-review rate is 42.5%, during the week starting September 01, 2025.

Because several points are outside the limits and there is a clear upward shift later in the period, the process is not in statistical control. The unusual weeks should be compared with patch notes, server outages, balance changes, bug reports, and cheating reports. Those records are not included in the Kaggle data, so this project does not claim one exact cause.

Table 5: Weeks outside the control limits

Week	Negative review rate	Result
2025-05-26	15.1%	Below LCL
2025-06-02	18.7%	Below LCL
2025-06-09	17.7%	Below LCL
2025-06-23	18.3%	Below LCL
2025-06-30	17.1%	Below LCL
2025-07-07	17.6%	Below LCL
2025-08-11	33.2%	Above UCL
2025-08-25	33.6%	Above UCL
2025-09-01	42.5%	Above UCL
2025-09-15	32%	Above UCL
2025-09-22	37.4%	Above UCL
2025-09-29	30.8%	Above UCL
2025-10-06	30.1%	Above UCL
2025-11-03	30.9%	Above UCL

The check sheet shows that Other/unclear is the largest category. Among the specific categories, gameplay/balance and technical bugs/performance are the two largest. These should receive the first improvement effort, while better complaint collection should reduce the unclear group.

4.1 Pareto Chart

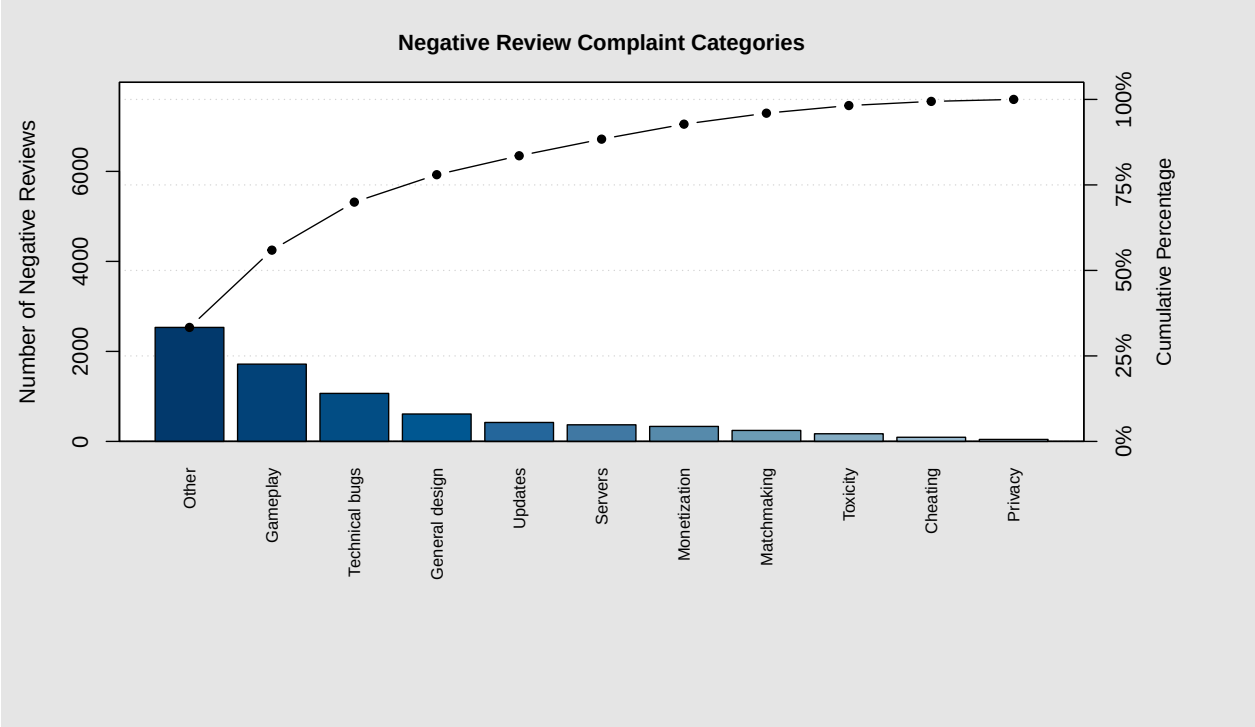


Figure 4: Pareto chart of negative-review complaint categories.

4.2 Chance and Assignable Causes

Chance causes include differences in player taste, random match results, normal changes in who writes reviews, and temporary internet problems on the player’s side. These create normal variation and cannot be fully removed.

Assignable causes include a bad patch, server outage, broken ability, large balance change, cheating exploit, paid content that does not work, or matchmaking failure. These are specific problems that the company may be able to find and correct.

4.3 Cause-and-Effect Diagram

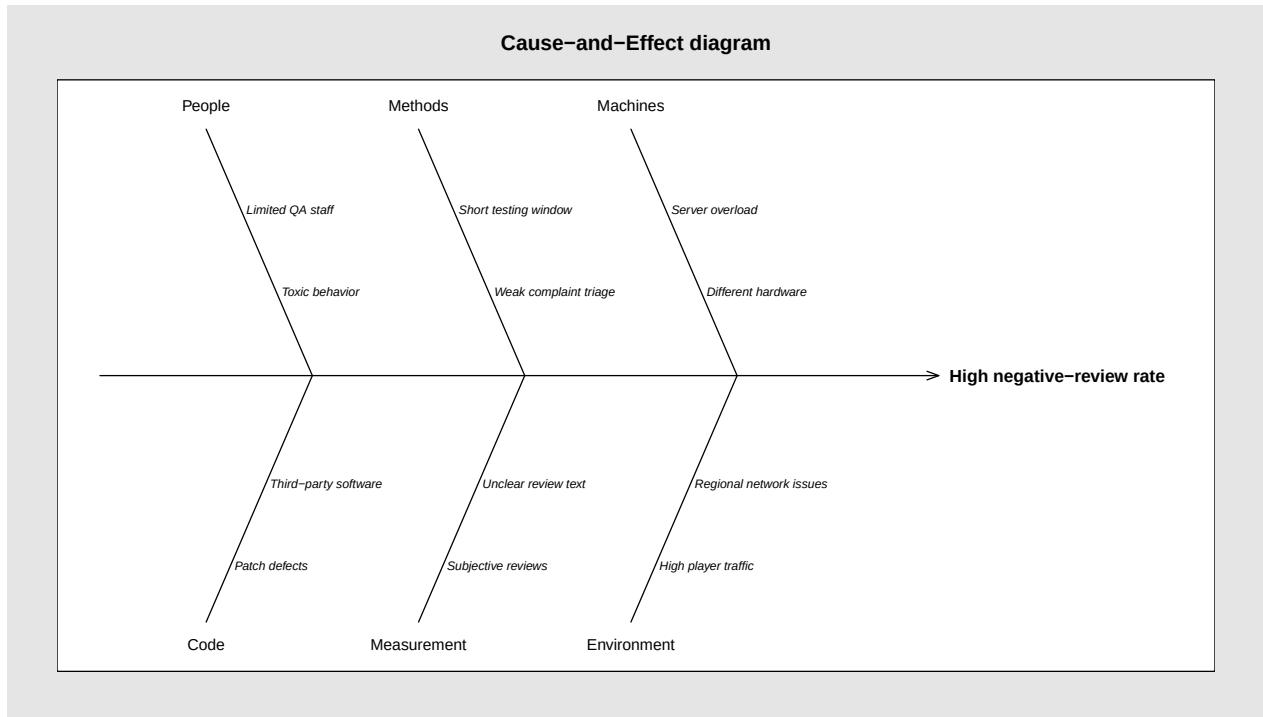


Figure 5: Cause-and-effect diagram for a high negative-review rate.

The Analyze tollgate is passed when the team reviews the out-of-control weeks, the Pareto chart, and the most likely assignable causes.

4.4 Defect Concentration Diagram

A defect concentration diagram is not useful for this dataset because the reviews do not consistently list a map, character, screen, platform, or physical location. A future complaint form should collect those fields. The team could then make a diagram showing where crashes, visual bugs, or gameplay problems are concentrated.

5 Improve

The first improvement is a standard in-game complaint form asking for platform, region, patch version, character, map, error type, and whether the issue can be repeated. This should reduce the large Other/unclear category and help developers reproduce problems faster.

The second improvement is a longer test period for major balance and performance updates. A small pilot group could test a patch before it reaches everyone. The team should compare crashes and disconnects across hardware, regions, and server conditions. Patch notes should also list known issues so players know the problem has been noticed.

5.1 Value Stream Map

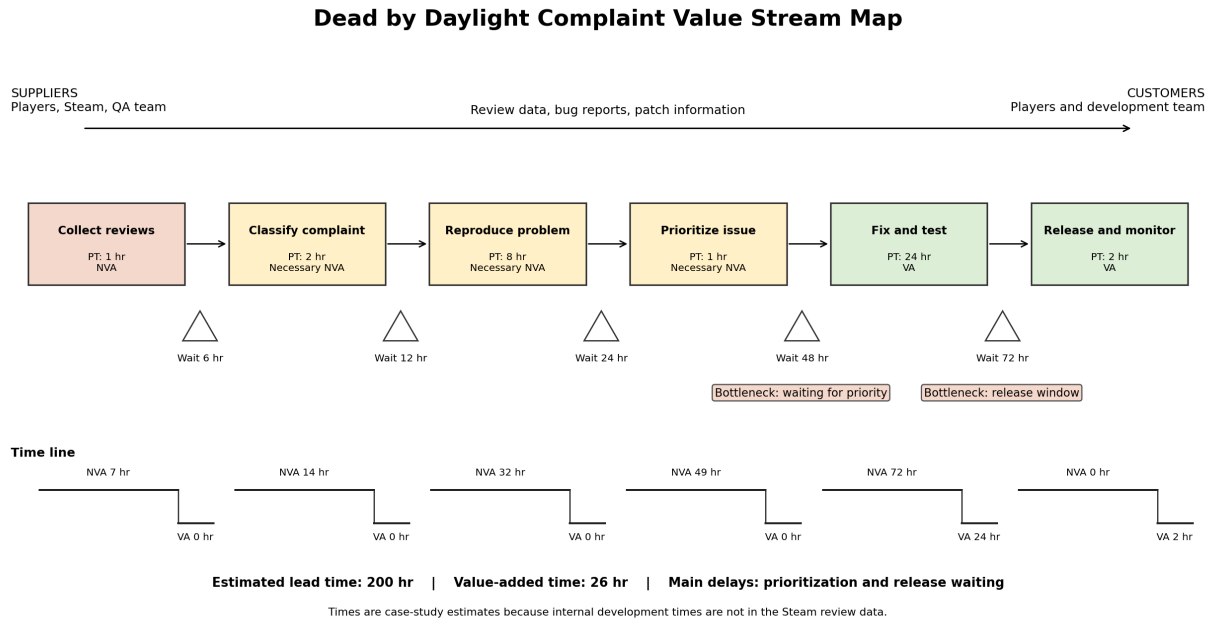


Figure 6: Estimated value stream map based on the provided template. Times are case-study assumptions.

The largest bottlenecks are the 48-hour wait for prioritization and the 72-hour wait for a release window. A 24-hour triage rule for serious complaints and an emergency hotfix path for critical defects would reduce those delays. The Improve tollgate is passed after owners are selected and the new complaint and pilot-release processes are tested.

6 Control

The team should update the p -chart each week. A point above the UCL should start an investigation within one business day. The analyst should verify the counts, read at least 50 reviews, compare the timing with patch and server records, and assign an owner. A point below the LCL should also be checked because it may show a real improvement worth keeping.

6.1 Out-of-Control Action Plan

Table 6: Out-of-control action plan

Signal	First action	Next step
Point above UCL	Check the counts and read at least 50 reviews	Compare patch and server records, then assign an owner
Point below LCL	Check for a data error or a real improvement	Record the helpful change and watch the next four weeks
Repeated upward pattern	Compare the main complaint categories	Escalate the problem before the next major release

6.2 Demerit System

Table 7: Demerit system

Class	Weight	Examples
A - Critical	100	Cannot launch; player progress is lost
B - Serious	50	Repeated crash or disconnect; serious cheating exploit
C - Major	10	Major balance or matchmaking issue; paid content is broken
D - Minor	1	Small visual or sound glitch; wording or translation error

The weights are larger for defects that stop people from playing or cause permanent loss. Class A gets 100 points, Class B gets 50, Class C gets 10, and Class D gets 1. The weekly score is:

$$\text{Score} = 100A + 50B + 10C + D$$

The Control tollgate is passed when the chart, demerit score, response owner, and four-week follow-up are part of the normal weekly process.

7 Conclusion

The review process was not in statistical control during the selected 30 weeks. The average negative-review rate was about 24.7%, and several weeks were outside the control limits. The Pareto analysis suggests that gameplay balance and technical quality should be worked on first. The large unclear category also shows that better complaint collection is needed.

Steam reviews are subjective and may mention more than one issue, so this project cannot prove the exact cause of every negative review. Even with that limitation, DMAIC gives the team a clear way to define the problem, measure weekly results, find unusual periods, test improvements, and keep monitoring after changes are made.

8 Appendix A: Data Availability

The review data were downloaded from [Steam Reviews English - Dead by Daylight on Kaggle](#).

The complaint categories are based on keyword rules, so some reviews may be placed in the wrong category. The dataset also does not include internal server logs, confirmed bug causes, or exact development times.

9 Appendix B: R Code

```
library(qcc)
library(SixSigma)
library(knitr)

knitr::opts_chunk$set(
  echo = FALSE,
  warning = FALSE,
  message = FALSE,
  fig.align = "center",
  out.width = "95%",
  fig.width = 8,
  fig.height = 5,
  fig.pos = "!ht"
)
reviews_full <- read.csv(
  "steam_reviews_381210_english.csv",
  stringsAsFactors = FALSE
)

reviews_full$date_created <- as.Date(
  substr(reviews_full$date_created, 1, 10)
)

reviews_full$sentiment <- as.numeric(reviews_full$sentiment)

reviews <- reviews_full[
  reviews_full$date_created >= as.Date("2025-04-14") &
  reviews_full$date_created <= as.Date("2025-11-09"),
]

if (nrow(reviews) == 0) {
  stop("No reviews were found in the selected date range.")
}

reviews$week <- as.Date(
  cut(
    reviews$date_created,
    breaks = "week",
    start.on.monday = TRUE
  )
)
```

```

weeks <- sort(unique(reviews$week))
total <- numeric(length(weeks))
negative <- numeric(length(weeks))

for (i in seq_along(weeks)) {
  one_week <- reviews[reviews$week == weeks[i], ]
  total[i] <- nrow(one_week)
  negative[i] <- sum(one_week$sentiment == 0, na.rm = TRUE)
}

weekly <- data.frame(
  week = weeks,
  total = total,
  negative = negative
)

weekly$p <- weekly$negative / weekly$total

pbar <- sum(weekly$negative) / sum(weekly$total)
weekly$ucl <- pbar + 3 * sqrt(pbar * (1 - pbar) / weekly$total)
weekly$lcl <- pbar - 3 * sqrt(pbar * (1 - pbar) / weekly$total)
weekly$lcl[weekly$lcl < 0] <- 0

weekly$status <- "In control"
weekly$status[weekly$p > weekly$ucl] <- "Above UCL"
weekly$status[weekly$p < weekly$lcl] <- "Below LCL"

out_of_control <- weekly[weekly$status != "In control", ]
above_count <- sum(weekly$status == "Above UCL")
below_count <- sum(weekly$status == "Below LCL")

negative_reviews <- reviews[reviews$sentiment == 0, ]
negative_reviews$review[is.na(negative_reviews$review)] <- ""
text <- tolower(negative_reviews$review)
negative_reviews$category <- "Other/unclear"

hit <- grepl(
  paste(
    "crash|fps|frame|stutter|performance|laggy|freeze|freezing|",
    "optimization|bug|broken|glitch|fix your game|",
    "doesn.?t work|not work",
    sep = ""
  ),
  text
)
negative_reviews$category[hit] <- "Technical bugs/performance"

hit <- grepl(
  "server|disconnect|connection|ping|latency|network|rubberband",

```

```

    text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Server/connection"

hit <- grepl(
  "matchmaking|queue|mmr|rank|ranked|skill.?based",
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Matchmaking"

hit <- grepl(
  paste(
    "balance|balanced|unbalanced|overpowered|underpowered|",
    "nerf|buff|ability|abilities|killer|survivor|camping|",
    "tunnel|hook|perk|loop|slug|chase",
    sep = ""
  ),
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Gameplay/balance"

hit <- grepl(
  "cheat|hacker|hack|exploit",
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Cheating"

hit <- grepl(
  paste(
    "dlc|money|price|expensive|pay.?to.?win|",
    "microtransaction|cash|purchase|cosmetic|skin|buy",
    sep = ""
  ),
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Monetization"

hit <- grepl(
  "privacy|data|eula|terms of service|personal information|consent",
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Privacy/data"

hit <- grepl(
  "update|patch|devs|developer|bhvr|behavior|removed|change|rework",
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Updates/developer decisions"

hit <- grepl(

```

```

    "toxic|community|teammate|team mate|grief|slur|racis|harass",
    text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "Toxicity/community"

hit <- grepl(
  paste(
    "not fun|no fun|boring|repetitive|bad game|worst game|",
    "hate|sucks|trash|terrible|horrible|unfun|",
    "waste of time|waste of money|not recommend",
    sep = ""
  ),
  text
) & negative_reviews$category == "Other/unclear"
negative_reviews$category[hit] <- "General enjoyment/design"

negative_reviews$actionable <-
  negative_reviews$category != "Other/unclear" &
  negative_reviews$category != "General enjoyment/design"

weekly$actionable <- 0

for (i in seq_along(weeks)) {
  one_week_negative <- negative_reviews[
    negative_reviews$week == weeks[i],
  ]

  weekly$actionable[i] <- sum(
    one_week_negative$actionable,
    na.rm = TRUE
  )
}

weekly$actionable_p <- weekly$actionable / weekly$total

counts <- sort(table(negative_reviews$category), decreasing = TRUE)

check_sheet <- data.frame(
  Complaint = names(counts),
  Count = as.numeric(counts)
)

check_sheet$Percent <- round(
  100 * check_sheet$Count / sum(check_sheet$Count),
  1
)

short_names <- c(
  "Other/unclear" = "Other",
  "Gameplay/balance" = "Gameplay",

```

```

"Technical bugs/performance" = "Technical bugs",
"General enjoyment/design" = "General design",
"Updates/developer decisions" = "Updates",
"Server/connection" = "Servers",
"Monetization" = "Monetization",
"Matchmaking" = "Matchmaking",
"Toxicity/community" = "Toxicity",
"Cheating" = "Cheating",
"Privacy/data" = "Privacy"
)

check_sheet$ShortName <- short_names[check_sheet$Complaint]
missing_name <- is.na(check_sheet$ShortName)
check_sheet$ShortName[missing_name] <- check_sheet$Complaint[missing_name]

pareto_counts <- check_sheet$Count
names(pareto_counts) <- check_sheet$ShortName

out_table <- data.frame(
  Week = format(out_of_control$week, "%Y-%m-%d"),
  `Negative review rate` = paste0(round(100 * out_of_control$p, 1), "%"),
  Result = out_of_control$status,
  check.names = FALSE
)

ctq_table <- data.frame(
  CTQ = c(
    "Weekly negative-review proportion",
    "Weekly actionable-complaint proportion"
  ),
  Definition = c(
    "Negative reviews divided by all reviews in the week",
    "Reviews naming a specific issue divided by all reviews in the week"
  ),
  Direction = c("Smaller is better", "Smaller is better")
)

sipoc_table <- data.frame(
  Part = c("Suppliers", "Inputs", "Process", "Outputs", "Customers"),
  Details = c(
    "Players, QA testers, developers, server provider, and Steam",
    "Reviews, bug reports, game code, server data, and patch schedule",
    "Collect, classify, reproduce, prioritize, fix, test, release, and monitor",
    "Resolved issues, stable patches, patch notes, and lower negative-review rate",
    "Current players, future players, support team, and development team"
  )
)

tollgate_table <- data.frame(

```

```

Stage = c("Define", "Measure", "Analyze", "Improve", "Control"),
Tollgate = c(
  "Problem, scope, CTQs, SIPOC, and process map are agreed on",
  "Dates and sentiment are checked; weekly data and check sheet are ready",
  "Unusual weeks and main complaint categories are reviewed",
  "Owners are assigned and proposed changes are tested",
  "The chart, demerit score, and response plan are used each week"
)
)

action_table <- data.frame(
  Signal = c("Point above UCL", "Point below LCL", "Repeated upward pattern"),
  `First action` = c(
    "Check the counts and read at least 50 reviews",
    "Check for a data error or a real improvement",
    "Compare the main complaint categories"
  ),
  `Next step` = c(
    "Compare patch and server records, then assign an owner",
    "Record the helpful change and watch the next four weeks",
    "Escalate the problem before the next major release"
  ),
  check.names = FALSE
)

demerit_table <- data.frame(
  Class = c("A - Critical", "B - Serious", "C - Major", "D - Minor"),
  Weight = c(100, 50, 10, 1),
  Examples = c(
    "Cannot launch; player progress is lost",
    "Repeated crash or disconnect; serious cheating exploit",
    "Major balance or matchmaking issue; paid content is broken",
    "Small visual or sound glitch; wording or translation error"
  )
)

kable(ctq_table, format = "pipe", caption = "CTQ characteristics")
kable(sipoc_table, format = "pipe", caption = "SIPOC summary")
steps <- c("COLLECT", "CLASSIFY", "REPRODUCE", "FIX AND TEST", "RELEASE")
inputs.overall <- c("Player reviews", "Bug reports", "Game and server data")
outputs.overall <- c("Stable patch", "Clear patch notes", "Lower negative-review rate")

input.output <- vector("list", length(steps))
input.output[[1]] <- c("Steam reviews")
input.output[[2]] <- c("Weekly review data")
input.output[[3]] <- c("Complaint categories")
input.output[[4]] <- c("Confirmed issue")
input.output[[5]] <- c("Tested fix")

x.parameters <- vector("list", length(steps))
x.parameters[[1]] <- list(c("Review date", "C"), c("Sentiment", "Cr"))

```

```

x.parameters[[2]] <- list(c("Keyword rule", "P"), c("Reviewer", "C"))
x.parameters[[3]] <- list(c("Platform", "C"), c("Patch version", "C"))
x.parameters[[4]] <- list(c("Developer", "C"), c("Test plan", "P"))
x.parameters[[5]] <- list(c("Release date", "C"), c("Server status", "N"))

y.features <- vector("list", length(steps))
y.features[[1]] <- list("Weekly data Cr")
y.features[[2]] <- list("Complaint category Cr")
y.features[[3]] <- list("Confirmed cause Cr")
y.features[[4]] <- list("Passed test Cr")
y.features[[5]] <- list("Lower negative rate Cr")

ss.pMap(
  steps,
  inputs.overall,
  outputs.overall,
  input.output,
  x.parameters,
  y.features,
  sub = "Dead by Daylight Review Project"
)

knitr::include_graphics("process_flowchart.png")
kable(tollgate_table, format = "pipe", caption = "DMAIC tollgates")
old_width <- getOption("width")
options(width = 10)

week_labels <- format(weekly$week, "%m/%d")

p_chart <- qcc(
  data = weekly$negative,
  type = "p",
  sizes = weekly$total,
  labels = week_labels,
  plot = FALSE
)

str(p_chart, max.level = 1, give.attr = FALSE)

options(width = old_width)
plot(
  p_chart,
  title = "Weekly Negative-Review Proportion",
  xlab = "Week",
  ylab = "Negative-review proportion",
  axes.las = 2,
  cex.axis = 0.75
)
kable(
  check_sheet[, c("Complaint", "Count", "Percent")],
  format = "pipe",

```

```

caption = "Complaint check sheet"
)
kable(out_table, format = "pipe", caption = "Weeks outside the control limits")
par(mar = c(7, 4.5, 3, 4.5))

invisible(
  pareto.chart(
    pareto_counts,
    main = "Negative Review Complaint Categories",
    ylab = "Number of Negative Reviews",
    ylab2 = "Cumulative Percentage",
    las = 2,
    cex.names = 0.72
  )
)
causes <- list(
  People = c("Limited QA staff", "Toxic behavior"),
  Methods = c("Short testing window", "Weak complaint triage"),
  Machines = c("Server overload", "Different hardware"),
  Code = c("Patch defects", "Third-party software"),
  Measurement = c("Subjective reviews", "Unclear review text"),
  Environment = c("High player traffic", "Regional network issues")
)

cause.and.effect(
  cause = causes,
  effect = "High negative-review rate",
  cex = c(0.75, 0.58, 0.82)
)

knitr::include_graphics("value_stream_map.png")
kable(action_table, format = "pipe", caption = "Out-of-control action plan")
kable(demerit_table, format = "pipe", caption = "Demerit system")

```